Advanced Guide — Deployment, Customization & Maintenance

This guide covers production deployment, Stripe webhook setup, customization patterns, adding new features, and troubleshooting.

Table of Contents

1. Production Deployment
2. Stripe Webhook Setup
3. Domain & DNS Setup
4. Customizing the Frontend
5. Adding New Pages
6. Adding a New API Route
7. Modifying the Cart Logic
8. Adding New Languages
9. Extending the WordPress Plugin
10. Shipping & Tax Configuration
11. Analytics & Tracking
12. Performance Optimization
13. Security Considerations
14. Troubleshooting
15. Architecture Decisions Explained

---

1. Production Deployment

Frontend → Vercel

The fastest way to deploy the Next.js frontend:

1. Push your code to GitHub/GitLab/Bitbucket
2. Go to https://vercel.com and sign in with your Git provider
3. Click **Add New** → **Project**
4. Select your repository
5. Add **Environment Variables** (same as `.env.local`):
    - `NEXT_PUBLIC_WOOCOMMERCE_URL`
    - `WOOCOMMERCE_CONSUMER_KEY`
    - `WOOCOMMERCE_CONSUMER_SECRET`
    - `STRIPE_SECRET_KEY`
    - `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`
    - `STRIPE_WEBHOOK_SECRET`
    - [illegible] set to your production URL

- `STRIPE_SECRET_KEY`
- `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`
- `STRIPE_WEBHOOK_SECRET`
- `NEXT_PUBLIC_SITE_URL` → set to your production URL
- `NEXT_PUBLIC_SITE_NAME`
- `NEXT_PUBLIC_GA_MEASUREMENT_ID` (optional)
- `NEXT_PUBLIC_META_PIXEL_ID` (optional)

6. Click **Deploy**

Every push to your main branch will automatically trigger a new deployment.

Backend → WordPress Hosting

Any WordPress host works. Recommended:

- **Cloudways** — Managed cloud hosting (DigitalOcean, AWS, Google Cloud)
- **WP Engine** — Managed WordPress hosting
- **SiteGround** — Budget-friendly

Ensure your WordPress host:

- Supports PHP 7.4+ (8.x recommended)
- Has SSL (HTTPS) enabled
- Has sufficient memory (256MB+ PHP memory limit)

CORS Configuration for Production

After deploying, update the allowed origins in your WordPress `functions.php`:

```
$allowed_origins = [
    'http://localhost:3000',           // Local development
    'https://peptivepeptides.com',     // Your production domain
    'https://www.peptivepeptides.com', // www variant
    'https://peptive.vercel.app',      // Vercel preview
];
```

2. Stripe Webhook Setup

Webhooks are how Stripe tells your site "a payment succeeded." This triggers order creation in WooCommerce.

Step 1: Create Webhook in Stripe Dashboard

1. Go to https://dashboard.stripe.com/webhooks
2. Click **Add Endpoint**
3. Endpoint URL: `https://your-site.com/api/webhooks/stripe`
4. Select events to listen to:
    - `checkout.session.completed` (most important)
    - `checkout.session.expired`

4. Select events to listen to:
   - `checkout.session.completed` (most important)
   - `checkout.session.expired`
   - `payment_intent.succeeded`
   - `payment_intent.payment_failed`
5. Click **Add Endpoint**
6. Copy the **Signing Secret** (starts with `whsec_`)

Step 2: Add Signing Secret to Environment

Add to your Vercel environment variables:

```
STRIPE_WEBHOOK_SECRET=whsec_your_signing_secret_here
```

Step 3: Test the Webhook

Use Stripe CLI for local testing:

```
# Install Stripe CLI
brew install stripe/stripe-cli/stripe  # Mac
# Or download from https://stripe.com/docs/stripe-cli

# Login
stripe login

# Forward webhooks to your local server
stripe listen --forward-to localhost:3000/api/webhooks/stripe

# In another terminal, trigger a test event
stripe trigger checkout.session.completed
```

How Webhooks Create WooCommerce Orders

```
Stripe payment succeeds
    │
    ▼
POST /api/webhooks/stripe
    │
    ├── Verify signature (stripe.webhooks.constructEvent)
    ├── Extract session details
    ├── Get line items from Stripe (with product metadata)
    ├── Separate products from tax/shipping line items
    ├── Map Stripe items to WooCommerce product IDs
    └── POST to WooCommerce REST API (/wc/v3/orders)
        │
        ├── Status: processing
        ├── set_paid: true
        ├── billing/shipping from checkout form
        ├── line_items with product_id and quantity
        └── meta_data with bundle_type, stripe_line_item_id
```

3. Domain & DNS Setup

Connecting Custom Domain to Vercel

3. Domain & DNS Setup

Connecting Custom Domain to Vercel

1. In Vercel: **Settings** → **Domains** → **Add Domain**
2. Add `peptivepeptides.com` (or your domain)
3. Vercel will show DNS records to add
4. In your domain registrar (GoDaddy, Namecheap, etc.):
   - Add an **A record** pointing to `76.76.21.21`
   - Add a **CNAME** for `www` pointing to `cname.vercel-dns.com`
5. Wait for DNS propagation (usually 5-30 minutes)

SSL

Vercel provides free SSL certificates automatically. WordPress hosting should also have SSL — most providers include Let's Encrypt for free.

---

4. Customizing the Frontend

Changing Colors

Edit `tailwind.config.ts`:

```
colors: {
  primary: {
    50: '#f0f9ff',
    // ... change these hex values
    900: '#0c4a6e',
  },
},
```

The primary color is used throughout via classes like `bg-primary-600`, `text-primary-500`, etc.

Changing Fonts

Edit `app/layout.tsx`:

```
// Change from Inter to any Google Font
import { Poppins } from 'next/font/google';

const poppins = Poppins({
  subsets: ['latin'],
  weight: ['400', '500', '600', '700'],
  variable: '--font-poppins'
});
```

Changing the Logo

Changing the Logo

Replace `/public/logo.avif` with your new logo. Update the reference in:

- `app/layout.tsx` (metadata icons)
- `components/layout/Header.tsx` (header logo)

Changing the Hero Banner Background

Replace `/public/banner.png` or update the URL in `app/page.tsx`:

```
style={{
  backgroundImage: "url('/your-new-banner.png')",
}}
```

Changing the WhatsApp Number

Edit `app/layout.tsx` — find `href="https://wa.me/9715582259 19"` and change the number.

Changing Countries in Checkout

Edit `app/checkout/page.tsx` — modify the `AVAILABLE_COUNTRIES` array at the top of the file.

Changing Shipping Rates

Edit `app/api/calculate-shipping-tax/route.ts`:

```
if (country === 'AE') {
  shippingCost = 0; // Free shipping in UAE
} else if (['SA', 'KW', 'QA', 'BH', 'OM'].includes(country)) {
  shippingCost = 25; // GCC
} else {
  shippingCost = 50; // International
}
```

Changing Tax Rate

Edit `app/checkout/page.tsx`:

```
const TAX_RATE = 0.05; // Change from 5% to your rate
```

5. Adding New Pages

Next.js uses file-based routing. To add a new page:

5. Adding New Pages

Next.js uses file-based routing. To add a new page:

Example: Adding an "About Us" Page

1. Create `app/about/page.tsx`:

```
export default function AboutPage() {
  return (
    <div className="px-6 sm:px-8 md:px-12 lg:px-12 xl:px-12 2xl:px-48 py-12">
      <h1 className="text-4xl font-bold text-gray-900 mb-8">About Us</h1>
      <p className="text-gray-600 text-lg">
        Your about us content here.
      </p>
    </div>
  );
}
```

2. Add it to navigation in `components/layout/Header.tsx`:

```
const navigation = [
  // ... existing items
  { name: 'About Us', href: '/about' },
];
```

The page is now available at `/about`.

---

6. Adding a New API Route

Example: Contact Form Endpoint

Create `app/api/contact/route.ts`:

```
import { NextRequest, NextResponse } from 'next/server';

export async function POST(request: NextRequest) {
  try {
    const { name, email, message } = await request.json();

    if (!name || !email || !message) {
      return NextResponse.json(
        { error: 'All fields are required' },
        { status: 400 }
      );
    }

    // Send email or save to database
    // ... your logic here

    return NextResponse.json({ success: true });
  } catch (error) {
```

```
    // Send email or save to database
    // ... your logic here

    return NextResponse.json({ success: true });
  } catch (error) {
    return NextResponse.json(
      { error: 'Failed to send message' },
      { status: 500 }
    );
  }
}
```

Call it from a component:

```
const response = await fetch('/api/contact', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name, email, message }),
});
```

7. Modifying the Cart Logic

The cart lives in `store/cartStore.ts` (Zustand).

Adding a Quantity Limit

```
addItem: (item) => {
  const items = get().items;
  const totalQuantity = items.reduce((sum, i) => sum + i.quantity, 0);

  if (totalQuantity >= 10) {
    alert('Maximum 10 items in cart');
    return;
  }

  // ... rest of addItem logic
},
```

Adding a Discount Field

Update `types/index.ts`:

```
export interface CartItem {
  // ... existing fields
  discountedPrice?: string;
}
```

Then use `item.discountedPrice || item.price` in calculations.

Then use `item.discountedPrice || item.price` in calculations.

8. Adding New Languages

Step 1: Add Translations

Edit `lib/translations.ts` and add a new language object:

```
export const translations = {
  en: { ... },
  ar: { ... },
  fr: {  // New: French
    announcement_bar: "Bienvenue chez Peptive",
    header: {
      home: "Accueil",
      all_peptides: "Tous les Peptides",
      // ... all translations
    },
    // ...
  },
};
```

Step 2: Update Language Context

Edit `contexts/LanguageContext.tsx`:

```
type Language = 'en' | 'ar' | 'fr';  // Add 'fr'
```

Step 3: Add Language Option to Selector

Edit `components/LanguageSelector.tsx`:

```
const languages = [
  { code: 'en' as const, label: 'English' },
  { code: 'ar' as const, label: 'العربية' },
  { code: 'fr' as const, label: 'Français' },  // Add French
];
```

9. Extending the WordPress Plugin

Adding a New Custom Field

To add a new product field (e.g., "Research Notes"):

Adding a New Custom Field

To add a new product field (e.g., "Research Notes"):

1. Add Field to Admin Tab

In `includes/class-bundle-product-type.php`, inside the Arabic translation tab (or create a new tab):

```
<p class="form-field">
    <label for="research_notes"><?php _e('Research Notes', 'peptive-bundles'); ?></label>
    <textarea name="research_notes" id="research_notes" rows="4" style="width: 100%;"><?php echo esc_
</p>
```

2. Save the Field

In the `save_all_custom_fields` method:

```
if (isset($_POST['research_notes'])) {
    update_post_meta($post_id, '_research_notes', sanitize_textarea_field($_POST['research_notes']));
}
```

3. Expose via API

In `includes/class-bundle-api.php`, add to `add_bundle_data_to_store_api_callback`:

```
'research_notes' => get_post_meta($product->get_id(), '_research_notes', true) ?: '',
```

4. Read in Frontend

In `lib/woocommerce.ts`, extract from extensions:

```
research_notes: extensions.research_notes || '',
```

10. Shipping & Tax Configuration

Current Setup

Region	Shipping	Tax
UAE (AE)	Free	5%

Region	Shipping	Tax
UAE (AE)	Free	5%
GCC (SA, KW, QA, BH, OM)	AED 25	5%
International	AED 50	5%

Tax Configuration

Tax is calculated on the frontend at a flat 5% rate. To change:

1. Edit `app/checkout/page.tsx`:

```
const TAX_RATE = 0.05; // Change this
```

2. Update the Stripe line item label in `app/api/create-checkout-session/route.ts`:

```
name: 'Tax (5%)', // Update label
```

For More Complex Shipping Rules

The current shipping logic is in `app/api/calculate-shipping-tax/route.ts`. For WooCommerce-managed shipping zones, the `app/api/shipping-locations/route.ts` endpoint already fetches zone data — you could extend the shipping calculation to use those zones.

---

11. Analytics & Tracking

Google Analytics 4

GA4 is loaded via `components/analytics/GoogleAnalytics.tsx`. Set the `NEXT_PUBLIC_GA_MEASUREMENT_ID` env variable to enable it.

E-commerce events tracked automatically:

- `page_view` — every page navigation
- `view_item` — product detail page
- `add_to_cart` — when a product is added
- `remove_from_cart` — when removed
- `view_cart` — when cart page loads
- `begin_checkout` — when checkout starts
- `purchase` — after successful payment

Meta Pixel

- `purchase` — after successful payment

Meta Pixel

Meta Pixel is loaded via `components/analytics/MetaPixel.tsx`. Set `NEXT_PUBLIC_META_PIXEL_ID` to enable.

Events: `PageView`, `ViewContent`, `AddToCart`, `InitiateCheckout`, `Purchase`.

Adding Custom Events

Use the tracking library:

```
import { trackEvent, trackMetaEvent } from '@/lib/tracking';

// Google Analytics
trackEvent({
  action: 'newsletter_signup',
  category: 'engagement',
  label: 'footer_form',
});

// Meta Pixel
trackMetaEvent('Lead', { content_name: 'Newsletter' });
```

12. Performance Optimization

Current Optimizations

- **Client-side cart** — No server round-trips for add/remove
- **Store API for products** — Public, no auth overhead
- **Proxy caching** — API proxy adds `Cache-Control: public, s-maxage=60`
- **Image optimization** — Next.js automatic image optimization
- **Code splitting** — Next.js automatic per-page code splitting
- **Dynamic imports** — WooCommerce client loaded dynamically on pages that need it

Further Optimizations

1. **Static product pages**: Convert product pages to static generation:

```
export async function generateStaticParams() {
  const products = await woocommerce.getProducts({ perPage: 100 });
  return products.map(p => ({ slug: p.slug }));
}
```

2. **ISR (Incremental Static Regeneration)**: Add revalidation to product fetches for automatic cache updates.
3. **CDN for WordPress images**: Use a CDN (CloudFlare, BunnyCDN) in front of your WordPress media.

3. **CDN for WordPress images**: Use a CDN (CloudFlare, BunnyCDN) in front of your WordPress media.

---

13. Security Considerations

What's Already Secure

- **Stripe Checkout** — Payment is handled entirely by Stripe. No card data touches your server.
- **Webhook verification** — Stripe signatures are verified before processing.
- **JWT tokens** — User authentication uses industry-standard JWT.
- **Environment variables** — Secrets are never committed to Git.
- **CORS whitelist** — Only allowed origins can make API requests.

Security Checklist for Production

- [ ] Use Stripe **live mode** keys (not test keys)
- [ ] Set up Stripe webhook with proper signing secret
- [ ] Update CORS origins to only include your production domain
- [ ] Use HTTPS everywhere (Vercel does this automatically)
- [ ] Keep WordPress, WooCommerce, and plugins updated
- [ ] Use strong passwords for WordPress admin
- [ ] Consider Cloudflare for DDoS protection
- [ ] Restrict WordPress admin access by IP if possible
- [ ] Remove `/api/test-woocommerce` and `/api/test-wordpress` routes in production

WooCommerce API Key Security

The Consumer Key/Secret are stored as **server-side environment variables** (not `NEXT_PUBLIC_`). They are never sent to the browser. They are only used in:

- API routes (server-side)
- `lib/woocommerce.ts` REST API client (server-side only)

---

14. Troubleshooting

"CORS Error" in Browser Console

Cause: WordPress is rejecting requests from your frontend domain.

Fix:

1. Make sure `UPDATED-FUNCTIONS.php` code is in your `functions.php`
2. Add your frontend URL to `$allowed_origins`
3. Clear any WordPress cache plugins

"Products Load on Server but Not in Browser"

Cause: The Store API proxy is not working.

Fix:

1. Check the `/api/wp-proxy/[...path]` route is

Cause: The Store API proxy is not working.

Fix:

1. Check that `NEXT_PUBLIC_WOOCOMMERCE_URL` is accessible
2. Visit `https://your-wp-site.com/wp-json/wc/store/v1/products` directly
3. Check the proxy route at `/api/proxy/wc-store/products`

"Stripe Checkout Creates Session but Payment Doesn't Create WooCommerce Order"

Cause: Webhook is not configured or failing.

Fix:

1. Check Stripe Dashboard → Webhooks → see if events are being received
2. Check the webhook endpoint URL is correct
3. Check webhook signing secret matches `STRIPE_WEBHOOK_SECRET`
4. Check Vercel function logs for errors

"JWT Login Returns 403"

Cause: JWT plugin not configured properly.

Fix:

1. Verify `JWT_AUTH_SECRET_KEY` is in `wp-config.php`
2. Add `.htaccess` rule to pass Authorization header:

```
RewriteEngine on
RewriteCond %{HTTP:Authorization} ^(.*)
RewriteRule ^(.*) - [E=HTTP_AUTHORIZATION:%1]
```

"Arabic Text Not Showing"

Cause: Plugin data not reaching frontend.

Debug:

1. Check product has Arabic fields filled in WP admin
2. Visit `https://your-wp-site.com/wp-json/wc/store/v1/products?slug=your-product`
3. Look for `extensions.peptive-bundles` in the response
4. If empty, the plugin might not be activated

"Build Fails on Vercel"

Cause: Usually missing environment variables or TypeScript errors.

Fix:

1. Check Vercel build logs for the exact error
2. Ensure all environment variables are set in Vercel
3. Run `pnpm build` locally to reproduce the error

1. Check Vercel build logs for the exact error
2. Ensure all environment variables are set in Vercel
3. Run `pnpm build` locally to reproduce the error

---

15. Architecture Decisions Explained

Why Zustand Instead of WooCommerce Cart API?

The WooCommerce Store API provides server-side cart management, but it requires:

- Cookie-based sessions (problematic across domains)
- Network round-trips for every add/remove
- Complex nonce/session handling

Zustand gives us:

- Instant cart operations (no network delay)
- Works perfectly cross-domain
- Persists to localStorage automatically
- No session management needed

The trade-off: Cart is client-side only, not synced with WooCommerce. Orders are created via Stripe webhook after payment.

Why Stripe Checkout Instead of WooCommerce Checkout?

WooCommerce's native checkout requires server-side rendering of PHP forms, which conflicts with headless architecture. Stripe Checkout provides:

- PCI-compliant hosted payment page
- Supports Apple Pay, Google Pay, Link
- No need to handle card data
- Automatic 3D Secure authentication
- Simple integration via API

Why Next.js API Proxy for Store API?

Browsers enforce CORS policy. Even with CORS headers on WordPress, some setups (especially with caching plugins) strip or misconfigure headers. The proxy route:

- Guarantees CORS is never an issue
- Adds caching headers
- Runs server-side, bypassing browser restrictions

Why Excel File for Verification Codes?

Simple and effective for the use case. The client manages verification codes in a spreadsheet. The API reads/writes directly. For larger scale, this should migrate to a database table.

Why Client-Side Product Fetching?

Product pages use `useEffect` + `useState` instead of server-side data fetching. This avoids build-time errors when the WordPress backend is unreachable and keeps the site deployable even without a running

Why Client-Side Product Fetching?

Product pages use `useEffect` + `useState` instead of server-side data fetching. This avoids build-time errors when the WordPress backend is unreachable and keeps the site deployable even without a running WooCommerce instance.

This is the advanced technical reference for the Peptive Peptides project. For basics, see the [Beginner's Guide](#).